

Overview: What is Architectural Design?

Architectural design is the first stage in the software design process. It acts as the critical link between requirements engineering and detailed system design. It involves identifying the major sub-systems that make up a system and establishing a framework for how they will be controlled and how they will communicate.

TOPIC 1: Architectural Design Decisions

Before drawing any diagrams, a software architect must make fundamental decisions that will profoundly affect the system. There is no single "right" architecture; it is a creative process.

1. Why is Architectural Design Important?

- Stakeholder Communication:** It is a high-level representation (often simple box-and-line diagrams) that non-technical managers and clients can easily understand.
- System Analysis:** It allows you to analyze early on if the system will meet non-functional requirements (like performance or security).
- Large-scale Reuse:** The architecture is often the same for systems with similar requirements, allowing entire frameworks to be reused.

2. Designing for System Attributes & Conflicts The architecture you choose depends entirely on which non-functional requirement is most important.

- Performance:** Localize critical operations to minimize communication. *Strategy: Use large-grain (big) components rather than fine-grain (small) ones.*
- Security:** *Strategy: Use a layered architecture*, placing the most critical assets in the innermost layers (like a bank vault inside a secure building).
- Safety:** *Strategy: Isolate safety-critical components* into a small number of sub-systems so they are easy to validate and protect.
- Availability:** *Strategy: Include redundant (backup) components* so the system can switch to a backup without stopping if a failure occurs.
- Maintainability:** *Strategy: Use fine-grain, self-contained components* so that changing one small part doesn't break the whole system.

The Catch (Architectural Conflicts): You cannot have everything perfectly. For example, using *large-grain components* improves performance (less communication delay) but hurts maintainability (harder to update). Introducing *redundant data* improves availability but creates multiple points of attack, making security much harder.

EXAM POINT GUIDE (40 Marks - Part B & C Focus)

- Frequent Question:** "Analyze why design conflicts might arise when designing an architecture where availability and security requirements are the most important." (Usually 5-8 marks).
 - How to Score:** Explicitly define Availability (needs redundancy/easy access) and Security (needs layers/restricted access). State that redundant components create multiple points of vulnerability, creating a direct conflict.
-

TOPIC 2: System Organization

System organization reflects the overall, high-level strategy used to structure a system into macro-level sub-systems.

1. The Repository Model (Shared Data)

- Simplified Concept:** Think of a massive library (the central database) surrounded by different students (the sub-systems). The students do not talk to each other; they only interact with the library.
- How it Works:** All shared data is held in a central database. Sub-systems access and modify this central data.
- Pros:** Efficient way to share large amounts of data; backup, security, and access control are centralized in one place.
- Cons:** A single point of failure (if the repository goes down, the whole system dies); all sub-systems must conform to one single data model.

- Diagram Directions:** Draw a large central rectangle labeled "Project Repository". Draw several smaller rectangles around it (e.g., "Design Editor", "Code Generator"). Draw double-sided arrows connecting the surrounding boxes directly to the central repository.

2. The Client-Server Model

- Simplified Concept:** Think of a restaurant. The customers (clients) request food, and the kitchen (server) provides the service.
- How it Works:** The system is organized as a set of services provided by servers, and clients that access those services over a network.
- Pros:** Easy to distribute across a network; servers can be added or upgraded without affecting clients.
- Diagram Directions:** Draw circles/rectangles at the top labeled "Client 1", "Client 2". Draw a horizontal line representing the "Network" below them. Below the network, draw rectangles labeled "Server A", "Server B" (e.g., Print Server, Database Server). Connect clients and servers to the network line.

3. The Layered (Abstract Machine) Model

- Simplified Concept:** Think of an onion. You have to peel back the outer layer to get to the next layer. The outermost layer is what the user touches; the innermost layer is the computer hardware.
- How it Works:** The system is organized into layers, where each layer provides a specific set of services to the layer *immediately above it*.
- Pros:** Supports incremental development (you can build layer by layer); if a layer's internal code changes, it doesn't break the whole system as long as the interface remains the same.
- Cons:** Performance suffers because a request must pass through multiple layers of interpretation.
- Diagram Directions:** Draw a stack of rectangles. Top layer: "Application / User Interface". Next layer: "System Management". Next layer: "Database Layer". Bottom layer: "Operating System". Arrows point top-down.

EXAM POINT GUIDE (40 Marks - Part B & C Focus)

- Frequent Question:** "Describe repository model with appropriate example. State advantages and disadvantages." (5-6 marks). Or, "Suggest most appropriate structural model for an automated robot floor cleaner / University Marks card." (8 marks).
- How to Score:** For specific systems, you *must* justify your choice.
 - University Marks Card:* Choose **Repository**, because it requires centralized, heavily shared student data.
 - Configuration Management / Robot Cleaner:* Choose **Layered**, because lower layers handle raw hardware/databases, while top layers handle specific application logic. Draw the diagram exactly as directed above.

TOPIC 3: Modular Decomposition Styles

Once the high-level system organization (from Topic 2) is chosen, those large sub-systems must be broken down into smaller, manageable **modules**.

•*Note the difference:* A **sub-system** is independent and can run on its own. A **module** is a smaller component that provides services to other modules and usually cannot run independently.

1. Object-Oriented Decomposition

- Simplified Concept:** Decomposing the system into a set of communicating objects that reflect real-world entities.
- How it Works:** Modules are designed as objects with their own private state (attributes) and defined operations (methods). Objects communicate by calling each other's services.
- Pros:** Objects map perfectly to real-world entities (like "Invoice", "Customer", "Receipt"), making the system easy to understand. They are loosely coupled, so implementation changes don't cause widespread damage.
- Cons:** Complex to manage interactions; if an object's interface changes, all objects calling it must be updated.
- Diagram Directions (UML Class Diagram style):** Draw rectangles divided into three horizontal sections. Top: Object Name (e.g., **Customer**). Middle: Attributes (e.g., **name, address**). Bottom: Operations (e.g., **updateAddress()**). Use dashed arrows to show which object calls which.

2. Function-Oriented Pipelining (Data-Flow Model)

- Simplified Concept:** Think of a factory assembly line. Raw materials go in, get processed at station 1, move to station 2, and exit as a finished product.
- How it Works:** The system is decomposed into functional modules that accept input data, process/transform it, and produce output data. It is widely used in batch-processing (like generating payrolls or monthly invoices).
- Pros:** Very intuitive flow; transformations can be reused easily; simple to implement sequentially.
- Cons:** Hard to use for interactive systems (like video games or ATMs) where events happen unpredictably.
- Diagram Directions:** Draw labeled rectangles representing the transformations (e.g., "Read records", "Compute Salary", "Print Receipt"). Connect them with solid arrows. Label the arrows with the specific data being passed (e.g., "Salary Data", "Invoice Details").

EXAM POINT GUIDE (40 Marks - Part B Focus)

- Frequent Question:** "Differentiate between the two strategies used for decomposing a Sub-system to modules." (6 marks).
- How to Score:** Create a comparison table. Contrast OO Decomposition (state-based, real-world entity mapping, loosely coupled interfaces) with Pipelining (data-driven, input-to-output transformations, sequential flow). Mention that pipelining is bad for interactive systems.

TOPIC 4: Control Styles

Structural models (like Layered or Repository) don't tell us *how* the execution of the system is controlled. We need a control model to coordinate when sub-systems run.

1. Centralized Control One sub-system acts as the "boss" and manages the execution of all other sub-systems.

•The Call-Return Model (For Sequential Systems):

- Simplified Concept:* A strict corporate hierarchy. The CEO gives a task to a Manager, who gives it to a Worker. The Worker finishes, reports back up to the Manager, who reports back to the CEO.
- How it Works:* Top-down subroutine model. The main program calls a routine, waits for it to finish, and gets control returned to it.
- Diagram Directions:* Draw a tree structure. Top rectangle: "Main Program". Arrows point down to middle rectangles: "Routine 1", "Routine 2". Arrows from those point to bottom rectangles: "Routine 1.1", etc..

•The Manager Model (For Concurrent Systems):

- Simplified Concept:* A conductor directing an orchestra where many instruments play at once.
- How it Works:* One central process coordinates the starting, stopping, and synchronization of other processes executing in parallel.

2. Event-Based Control Control is driven by externally generated events rather than a central "boss".

•Broadcast Models:

- Simplified Concept:* A radio station broadcasts a message: "Free pizza in the lobby!" Only the people who care about pizza will get up and respond.
- How it Works:* An event is broadcast to all sub-systems. Any sub-system that has registered an interest in that event will independently respond to it. Excellent for systems where independent tools from different vendors must work together.

•Interrupt-Driven Models (For Real-Time Systems):

- Simplified Concept:* You are doing homework, and the fire alarm goes off. You instantly drop your homework (interrupt) and run outside (handler).
- How it Works:* Used in real-time systems requiring split-second responses. Hardware external interrupts are detected by an interrupt vector, which instantly forces the system to transfer control to a specific handler.
- Diagram Directions:* Draw a series of downward vertical arrows labeled "Interrupts" at the very top. These point to a divided horizontal rectangle labeled "Interrupt Vector". Arrows point from the vector slots to individual rectangles labeled "Handler 1", "Handler 2", which then point down to ovals labeled "Process 1", "Process 2".

EXAM POINT GUIDE (40 Marks - Part C Focus)

•**Frequent Question:** "Suggest an appropriate control model for: 1) A system to control a Television unit, 2) A batch processing system that prints salary slips, 3) A system that monitors sugar levels and dispenses insulin." (Usually 8-10 marks).

•**How to Score (Crucial Justifications):**

- Television unit / Car Remote:* **Interrupt-Driven Control.** *Justification:* It is a real-time system that must respond instantly and asynchronously to human button presses.

- Batch processing / Salary Slips:* **Centralized Call-Return Model.** *Justification:* It is a predictable, sequential process where data flows step-by-step from input to calculation to printing.

- Sugar level / Insulin pump:* **Manager Model or Interrupt-Driven.** *Justification:* It requires continuous real-time polling of hardware sensors with strict, safety-critical timing constraints to inject insulin. Draw the diagram for the model you choose and write 2-3 points justifying it.